



Shiftbase QA Assessment

End-to-end automation of the signup & login flows

Both flows automated to the dashboard — with a handful of observations the tests surfaced on the demo environment along the way.

Playwright + TypeScript, on the element-interactions Steps API. Runs green from a cold clone with no configuration.

The brief, and what shipped

Automate two flows on the Shiftbase demo environment: signup + onboarding up to the dashboard, and login. This is what came back.



Both flows, end to end

Signup walks the 8-step questionnaire and setup wizard to the dashboard; login handles the MFA interstitial. Interactions read as intent through the Steps API — no raw selectors in any test.



Some observations along the way

Testing a real product surfaces things. A few came up while building the happy paths, and a focused exploratory pass found more — all reproduced, none false positives. Shared constructively, for the team to weigh.



Evidence, not claims

Each observation carries a reproduction and a screen recording or screenshot. Anything that couldn't be pinned by a portable test is written up with the reason, not forced into one.



Runnable by anyone

`npm ci && npm test` on a clean clone with no `.env`. Verified by cloning fresh; the suite provisions its own account through the real signup flow.

Coverage at a glance

28 tests across three Playwright projects: desktop, mobile, and an account-provisioning setup project.

23

passing tests

5

failing by design

6

pages mapped

56

elements

10

observations



The 5 failing tests fail on purpose

Each is tagged `@known-defect` and describes the behaviour the product is expected to have — so it fails today and would turn green once addressed, becoming a regression guard. `npm run test:no-defects` is 23 / 23 green.



Not weakened to pass

Rewriting a test to agree with unexpected behaviour would make the suite unable to catch it. The failing tests were left describing the expected behaviour instead.

What each test protects

Chosen, not sprayed — every scenario covers a distinct way a flow can break.

FLOW	COVERAGE
Signup	Registration → questionnaire → wizard → dashboard · form handoff · mobile viewport · empty-submit & per-field validation (email, phone, weak password, terms)
Login	Valid credentials · the MFA-promote interstitial · wrong password · unknown email · identical rejection message (no account enumeration) · protected-route redirect · “Back to login” ends the session · concurrent & stale sessions
Session	Survives a reload (asserts the dashboard re-renders, not just the URL) · primary navigation present

Deliberately out of scope, and noted in the README: everything past the dashboard, password reset, cross-browser, visual regression, load.

What the tests surfaced

Ten observations on the demo environment, each reproduced and independently re-checked. Shared as the kind of thing thorough automation is built to catch.

Highest priority · consent

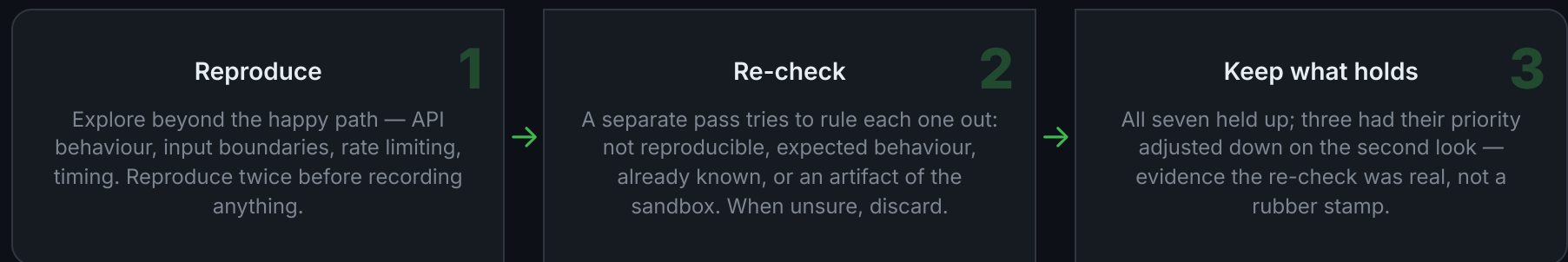
Terms & Data-Processing-Agreement acceptance is enforced only in the browser

On the demo environment, `POST /api/signup` accepts a request with `accept_conditions: false` and creates a usable account — the consent checkbox the UI requires has no server-side counterpart. Worth a look because the same pattern in production would record consent that was never given. Reported rather than automated, so a regression run never repeats it against the shared environment.

SOURCE	OBSERVATIONS	COVERAGE
While building	Blank page on an unrecognised modal route · duplicate email accepted, then its password rejected at login · no length cap on the name field	2 of 3 as tests
Exploratory pass	Consent bypass · no login rate-limit · timing enumeration · gateway error shown as bad credentials · swallowed reset message · fractional employee count · missing Back button	1 as test, 6 reported

How each observation was verified

Nothing rests on a single look — each was reproduced, then re-checked by an independent pass whose job was to disprove it.



✓ Selectors that survive copy changes

The questionnaire's radio inputs aren't directly clickable (`sr-only`); options are targeted by their stable `formcontrolname + value`, not the visible label — which can differ from the value.

✓ Unexpected behaviour reported, tests left honest

Where the app behaved unexpectedly, the test was left describing the expected behaviour and the observation was written up — never adjusted to match.

Engineering choices worth a look

The decisions a reviewer running this cold would notice.



No committed credentials

A setup project provisions an account via the real signup flow when no `.env` is present, so a cold clone runs green. Secrets live in `.env` (gitignored); an independent sweep confirmed none shipped.



Flake, root-caused not retried

The Angular questionnaire can drop clicks mid-render; every input is re-applied until it lands (verified 12× with retries off). The residual is the environment's own gateway variability, left to configured retries.



Watchable headed runs

`npm run test:headed` drops to one worker and paces each action, so a reviewer can actually follow the flows rather than watch windows race.



TLS reality documented

The demo serves a private-CA cert that rolls every 24h; the suite sets `ignoreHTTPSErrors` and says so plainly — on a production target that setting would hide a real issue.

Ready to run, honest about its edges

The README carries the full reasoning, the observations, and a candid “what I’d do next.”



What I'd do next

Flag the server-side consent check first — a small change with outsized value. Move account provisioning to the API to cut runtime. Add teardown for the accounts each run leaves behind.

@playwright/test

1.62

@civitas-cerebrum/element-interactions

0.3.9

TypeScript

strict